# Planejamento Lógico TaskTracker

## 1. Apresentação e Entendimento do Problema

| | |
|---|---|
| **Nome Completo:** | Lourenço Angelice Bettencourt do Canto Moniz |
| **Curso:** | Análise e Desenvolvimento de Sistemas |
| **E-mail Institucional:** | lourencocantomoniz@sempreceub.com |

### O Problema a Ser Resolvido

No cotidiano dinâmico de estudantes e profissionais, a ausência de um sistema estruturado de gerenciamento de tarefas leva à desorganização, esquecimento de prazos cruciais e sobrecarga mental. Quando os compromissos são anotados em papéis soltos ou mantidos apenas na memória, torna-se difícil priorizar atividades urgentes, acompanhar o progresso diário e manter o foco nas demandas mais importantes. Isso resulta em perda de produtividade, estresse e atrasos frequentes em entregas.

### A Solução Proposta (TaskTracker)

O TaskTracker é uma aplicação de gerenciamento de tarefas desenvolvida para resolver esse gargalo organizacional. O sistema permitirá ao usuário registrar suas tarefas de forma rápida com critérios de priorização (Alta, Média, Baixa), visualizar claramente a lista de compromissos pendentes, marcar tarefas como concluídas e acompanhar estatísticas simples de produtividade (como o total de tarefas finalizadas). O objetivo principal é fornecer uma ferramenta simples, intuitiva e confiável para centralizar a rotina diária.

## 2. Decomposição do Problema e Dados do Sistema

### Entradas de Dados

- **Título da Tarefa:** Texto simples que descreve resumidamente a ação (obrigatório).
- **Descrição da Tarefa:** Texto detalhado explicando o objetivo da tarefa (opcional).
- **Prioridade:** Nível de urgência da tarefa, restrito a três opções: Alta, Média ou Baixa.
- **Data Limite (Deadline):** Data final para conclusão da tarefa no formato DD/MM/AAAA.

## Saídas de Dados

- **Lista de Tarefas Organizadas:** Exibição estruturada das tarefas cadastradas, ordenadas por prioridade ou status.

- **Mensagens de Confirmação e Alerta:** Notificações na tela confirmando o cadastro bem-sucedido, alterações de status ou erros de validação.

- **Resumo de Desempenho:** Contador exibindo o número total de tarefas cadastradas, pendentes e concluídas.

## Regras do Sistema (Regras de Negócio)

- **Obrigatoriedade do Título:** O sistema não permite o cadastro de tarefas sem título ou apenas com espaços em branco.

- **Validação de Prioridade:** O campo de prioridade aceita exclusivamente os valores "Alta", "Média" ou "Baixa". Qualquer outro valor inserido pelo usuário deve ser rejeitado.

- **Integridade do Histórico:** Tarefas marcadas como concluídas não são excluídas permanentemente de imediato; elas mantêm seu registro arquivado para contabilização das estatísticas de produtividade.

# 3. Receita Lógica do Sistema

## Passo a Passo do Cadastro de Tarefa

1. Iniciar o módulo de cadastro de tarefa.
2. Solicitar ao usuário o Título da tarefa.
3. Verificar se o título digitado está vazio:
    - Se estiver vazio, exibir a mensagem de erro: "Erro: O título é obrigatório." e solicitar a digitação novamente.
4. Solicitar a Descrição da tarefa (opcional).
5. Solicitar a Prioridade (Alta, Média ou Baixa).
6. Verificar se a prioridade informada é válida (Alta, Média ou Baixa):
    - Se for inválida, exibir mensagem de erro e pedir para selecionar uma opção válida.
7. Solicitar a Data Limite para a entrega.
8. Armazenar os dados preenchidos na lista central de tarefas com o status inicial definido como "Pendente".
9. Exibir a mensagem de confirmação: "Sucesso: Tarefa cadastrada com êxito!".

### Passo a Passo da Visualização de Tarefas

1. Iniciar o módulo de exibição de tarefas.
2. Verificar se a lista de tarefas contém registros:
    - Se a lista estiver vazia, exibir a mensagem: "Nenhuma tarefa cadastrada no momento."
3. Se existirem tarefas cadastradas:
    - Percorrer cada item da lista.
    - Exibir na tela as informações de cada tarefa: ID, Título, Prioridade, Data Limite e Status (Pendente/Concluída).
4. Exibir o resumo contendo o número total de tarefas pendentes e concluídas.

# 4. Mapeamento Arquitetural para Git e GitHub

### Desenho da Árvore do Repositório

```
tasktracker-projeto/
├── README.md                 # Apresentação do projeto e instruções
de execução
├── .gitignore                # Arquivo para ignorar arquivos
temporários e de ambiente
├── docs/
│   └── planejamento_logico.pdf # Documento de planejamento lógico
(Fase 1)
└── src/
    └── main.py               # Código-fonte principal da aplicação em
Python
```

### Plano de Aprendizado para Git e GitHub (Fase 2)

Para a Fase 2 do projeto, pretendo dominar os fundamentos do controle de versão com Git e GitHub, compreendendo na prática como realizar commits atômicos e bem documentados para registrar o histórico de evolução do código. Meu objetivo é aprender a criar e gerenciar branches isoladas para desenvolver novas funcionalidades sem impactar a versão principal, além de praticar a resolução de conflitos de fusão (merge conflicts). Também almejo dominar a publicação e organização do repositório remoto no GitHub, garantindo uma documentação clara no README.md e mantendo um fluxo de trabalho profissional alinhado às boas práticas da engenharia de software.

# 5. Link da Apresentação em Vídeo

**URL do Vídeo:** https://youtu.be/Sz0hasjQ12U